CS772 Project Report

Understanding and Reproducing BayesVLM for Uncertainty Estimation in Vision-Language Models

Deepak Chaurasia *220330* **Riya Mittal** *220901* **J Kartik Sai** *241280001*

Link to GitHub

Abstract

Vision-language models such as CLIP and SigLIP have been very effective in classification, retrieval, and generation in image-text alignment. However, their deterministic nature renders them unreliable in real-world applications where confidence understanding of the model is crucial. Here, we learn and apply the method described in *Post-hoc Probabilistic Vision-Language Models* by Baumann et al., which introduces BayesVLM—a post-hoc, training-free uncertainty estimation method for pre-trained VLMs according to a Laplace approximation over the final projection layers. The method is augmented with an analytical form, ProbCosine, for approximating the distribution over cosine similarities. Our goal was to achieve in-depth understanding of the method, reproduce important experimental results, and explore enhancements or optimizations to the framework. Based on this work, we assess the efficacy of BayesVLM in providing better uncertainty calibration and enabling more informed data selection in active learning pipelines.

1 Problem Description / Motivation

Pre-trained vision-language models (VLMs) like CLIP and SigLIP excel at aligning text and images in a shared embedding space, but lack inherent uncertainty estimates-limiting their reliability in high-stakes domains and under distribution shift. To address this, our project builds on the BayesVLM framework, which originally used a Laplace approximation for post-hoc uncertainty quantification. We extend this by replacing Laplace with a scalable mean-field Variational Inference (VI) approach, allowing for efficient, mini-batch optimization and potentially better-calibrated uncertainty estimates. VI approximates the intractable Bayesian posterior with a simpler, parameterized variational distribution by solving an optimization problem that minimizes the Kullback-Leibler (KL) divergence to the true posterior. Unlike Laplace, which relies on a local Gaussian approximation around the MAP estimate and requires computation of the full Hessian, VI can be efficiently optimized using stochastic mini-batch gradients (e.g., Adam), making it scalable to large models and datasets. Moreover, VI is flexible and can better capture the true posterior's shape, often resulting in improved calibration and more reliable uncertainty estimates, especially in complex or high-dimensional settings.

Additionally, we improve the active learning pipeline by substituting k-nearest neighbors (k-NN) support set selection with Shared Nearest Neighbor (SNN) clustering. SNN enhances robustness to outliers and promotes diversity by selecting support exemplars from densely connected regions in the embedding space, ensuring better coverage and reducing redundancy. The sentiment behind replacing support set creation from KNN to SNN is to increase robustness against outliers and promote diversity. SNN utilises a shared-nearest-neighbor strategy to check the overlap in each point's k-nearest-neighbor lists, forming a graph where only pairs that share enough common

neighbors are linked. Our work reproduces and validates the original BayesVLM results, while empirically evaluating these methodological improvements in uncertainty estimation and active learning performance.

2 Literature Review and Description of Prior Work

BayesVLM adds to any static vision-language model (e.g., CLIP or SigLIP) a light post-hoc Laplace approximation over the posterior to its terminal linear layer, thus enabling the learning of predictive uncertainty without incurring additional training or costly inference methods. Precisely:

Frozen backbone: All the image and text encoders remain unchanged.

Last-Layer Laplace: We approximate a Gaussian posterior over the last projection layer's weights by calculating the (block-diagonal) Hessian of the negative log-likelihood at the MAP estimate(which was earlier calculated in point estimate).

Posterior approximation: This Laplace posterior on last-layer weights allows us to sample weights at test time, producing an ensemble of paired image–text embeddings for uncertainty estimation.

Efficiency: No model retraining, no Monte Carlo dropout or large ensembles—just a single Hessian pass and inexpensive Gaussian sampling.

By restricting the Bayesian approximation to the terminal layer, BayesVLM is made scalable to very large VLMs and yet still yields calibrated confidence scores for downstream tasks like zero-shot classification, retrieval, and generation.

2.1 Methodology Proposed in Paper

2.1.1 Laplace Approximation

Let $\boldsymbol{P} \in \mathbb{R}^{d \times d_{\text{IMG}}}$ and $\boldsymbol{Q} \in \mathbb{R}^{d \times d_{\text{TXT}}}$ be the projection matrices. The posterior distribution is approximated as:

$$p(\boldsymbol{\theta}|\mathcal{D}) \approx \mathcal{N}\left(\boldsymbol{\theta}_{\text{pre}}, \boldsymbol{H}^{-1}\right) \tag{1}$$

where $\boldsymbol{H} = -\nabla^2_{\boldsymbol{\theta}} \log p(\mathcal{D}|\boldsymbol{\theta})$ is the Hessian. For Kronecker-factored approximation:

$$\boldsymbol{H}_{\text{IMG}} \approx \left(\sqrt{\tau}\boldsymbol{A}_{\text{IMG}} + \sqrt{\lambda}\boldsymbol{I}\right) \otimes \left(\sqrt{\tau}\boldsymbol{B}_{\text{IMG}} + \sqrt{\lambda}\boldsymbol{I}\right) \tag{2}$$

$$\boldsymbol{H}_{\text{TXT}} \approx \left(\sqrt{\tau}\boldsymbol{A}_{\text{TXT}} + \sqrt{\lambda}\boldsymbol{I}\right) \otimes \left(\sqrt{\tau}\boldsymbol{B}_{\text{TXT}} + \sqrt{\lambda}\boldsymbol{I}\right) \tag{3}$$

2.1.2 Probabilistic Embeddings

Image and text embeddings become Gaussian distributions:

$$p(\boldsymbol{g}|\mathcal{D}) = \mathcal{N}\left(\boldsymbol{P}_{\text{MAP}}\phi(\boldsymbol{x}^{\text{IMG}}), \left(\phi(\boldsymbol{x}^{\text{IMG}})^\top \widetilde{\boldsymbol{A}}^{-1}_{\text{IMG}} \phi(\boldsymbol{x}^{\text{IMG}})\right) \widetilde{\boldsymbol{B}}^{-1}_{\text{IMG}}\right) \tag{4}$$

$$p(\boldsymbol{h}|\mathcal{D}) = \mathcal{N}\left(\boldsymbol{Q}_{\text{MAP}}\psi(\boldsymbol{x}^{\text{TXT}}), \left(\psi(\boldsymbol{x}^{\text{TXT}})^\top \widetilde{\boldsymbol{A}}^{-1}_{\text{TXT}} \psi(\boldsymbol{x}^{\text{TXT}})\right) \widetilde{\boldsymbol{B}}^{-1}_{\text{TXT}}\right) \tag{5}$$

2.1.3 ProbCosine Approximation

The cosine similarity distribution is approximated through:

$$\mathbb{E}[\text{S}_{\cos}(\boldsymbol{g}, \boldsymbol{h})] \approx \frac{\sum_{i=1}^{d} \mu_{\boldsymbol{g},i}\mu_{\boldsymbol{h},i}}{\sqrt{\sum_i(\mu^2_{\boldsymbol{g},i} + \sigma^2_{\boldsymbol{g},i})}\sqrt{\sum_i(\mu^2_{\boldsymbol{h},i} + \sigma^2_{\boldsymbol{h},i})}} \tag{6}$$

$$\text{Var}[\text{S}_{\cos}(\boldsymbol{g}, \boldsymbol{h})] = \frac{\sum_i \sigma^2_{\boldsymbol{g},i}(\sigma^2_{\boldsymbol{h},i} + \mu^2_{\boldsymbol{h},i}) + \sigma^2_{\boldsymbol{h},i}\mu^2_{\boldsymbol{g},i}}{\left(\sum_i \mu^2_{\boldsymbol{g},i} + \sigma^2_{\boldsymbol{g},i}\right)\left(\sum_i \mu^2_{\boldsymbol{h},i} + \sigma^2_{\boldsymbol{h},i}\right)} \tag{7}$$

Algorithm 1 Turn VLM into BayesVLM

Require: VLM encoders ϕ, ψ, training data $\mathcal{D}$

1: **for** each encoder Enc $\in \{\text{Img}, \text{Txt}\}$ **do**
2: Compute $\boldsymbol{A}_{\text{Enc}}$ via Eq. (22)
3: Compute $\boldsymbol{B}_{\text{Enc}}$ via Eq. (23)
4: **end for**
5: Optimize λ via marginal likelihood maximization
6: Update Kronecker factors $\widetilde{\boldsymbol{A}}, \widetilde{\boldsymbol{B}}$ **return** BayesVLM parameters

Algorithm 2 Compute Predictions

Require: BayesVLM model, image-text pair $(\boldsymbol{x}^{\text{IMG}}, \boldsymbol{x}^{\text{TXT}})$

1: Compute $\boldsymbol{\mu}_g, \boldsymbol{\Sigma}_g$ using Eq. (6)
2: Compute $\boldsymbol{\mu}_h, \boldsymbol{\Sigma}_h$ using Eq. (6)
3: Calculate $\mathbb{E}[\text{S}_{\cos}]$ via Eq. (7)
4: Calculate $\text{Var}[\text{S}_{\cos}]$ via Eq. (8)
5: Apply probit approximation **return** Softmax probabilities

2.1.4 Uncertainty-Aware Inference

Bayesian model averaging for classification:

$$p(y|\boldsymbol{x}) = \int \sigma(\boldsymbol{z}_i^\top \boldsymbol{z}_j) p(\boldsymbol{W}|\mathcal{D}) d\boldsymbol{W} \tag{8}$$

Active learning acquisition function:

$$\boldsymbol{x}^* = \arg\max_{\boldsymbol{x}} \text{Tr}(\boldsymbol{\Sigma}(\boldsymbol{x})) \tag{9}$$

3 Novelty of the work done

3.1 Support Set Selection: From KNN to SNN

The original BayesVLM active-learning pipeline uses **k-nearest neighbors (KNN)** to form the support set: for each test embedding, it finds the k closest training embeddings (by expected cosine similarity or 2-Wasserstein distance) and iteratively ensures k distinct neighbors per test point. This guarantees that every query has a directly relevant set of support examples.

Limitations of KNN.

- *Redundancy in dense regions:* KNN may select many highly similar points when the embedding space is crowded, reducing overall diversity.
- *Sensitivity to outliers:* every point, including noise or rare edge cases, is included, potentially lowering the informativeness of the support set.

Shared Nearest-Neighbor (SNN) Clustering. SNN builds a graph where two training points are connected only if they share at least m common neighbors in their individual k-NN lists. Clusters emerge as connected components of this graph, and one exemplar per component (e.g., the member closest to the centroid) is chosen as a support candidate.

Benefits of SNN.

- *Robustness to noise:* points with few shared neighbors are naturally excluded as outliers.
- *Enhanced diversity:* exemplars cover all dense regions of the embedding space, avoiding redundancy.
- *Adaptive to varying densities:* SNN implicitly adjusts to clusters of different shapes and scales without a single radius parameter.

Motivating Sentiment. By replacing per-test KNN with global SNN clustering for support-set creation, we prioritize *robustness* (ignoring uninformative outliers) and *diversity* (ensuring broad coverage of the data manifold), while still leveraging local neighborhood structure to target relevant regions in the vision-language embedding space.

3.2 From Laplace to Mean–Field Variational Inference

In this section, we replace the post-hoc Laplace approximation over the projection weights with a mean–field variational inference (MF-VI) approach. We begin by recalling the Laplace posterior, introduce our fully factorized Gaussian variational family(Although,we are ignoring intra-correlation between the entries of P and Q), derive the Evidence Lower Bound (ELBO), describe its optimization via the reparameterization trick, and finally show how this uncertainty propagates to the joint embeddings and cosine-similarity.

where Θ denotes the set of model parameters (here, the projection matrices P, Q), and H is the Hessian of the negative log joint at the MAP.

3.2.1 Mean–Field Variational Family

Rather than a single full-covariance Gaussian over all of P and Q, we assume a fully factorized ("mean–field") Gaussian posterior(but we are ignoring correlation in each wt in P and Q). Concretely, we did this

$$
\begin{aligned}
q(P) &= \prod_{u=1}^{d} \prod_{i=1}^{d_{\mathrm{IMG}}} \mathcal{N}\big(P_{u,i} \mid m_{u,i},\, s_{u,i}^2\big), \\
q(Q) &= \prod_{u=1}^{d} \prod_{j=1}^{d_{\mathrm{TXT}}} \mathcal{N}\big(Q_{u,j} \mid n_{u,j},\, t_{u,j}^2\big).
\end{aligned}
\tag{10}
$$

where

P The *image projection* matrix of dimension $P \in \mathbb{R}^{d \times d_{\mathrm{IMG}}}$, mapping the image encoder outputs $\phi(x) \in \mathbb{R}^{d_{\mathrm{IMG}}}$ to the joint space of d.

Q The *text projection* matrix of dimension $Q \in \mathbb{R}^{d \times d_{\mathrm{TXT}}}$, mapping the output of the text encoder $\psi(x) \in \mathbb{R}^{d_{\mathrm{TXT}}}$ to the same d-dimensional space.

$P_{u,i}$ The scalar entry in the u-th row and i-th column of P, for $u = 1, \ldots, d$ and $i = 1, \ldots, d_{\mathrm{IMG}}$.

$Q_{u,j}$ The scalar entry in the u-th row and j-th column of Q, for $u = 1, \ldots, d$ and $j = 1, \ldots, d_{\mathrm{TXT}}$.

$m_{u,i}$ The *variational mean* of $P_{u,i}$; grouped into a mean matrix $M \in \mathbb{R}^{d \times d_{\mathrm{IMG}}}$.

$s_{u,i}$ The *variational standard deviation* of $P_{u,i}$; grouped into a matrix $S \in \mathbb{R}^{d \times d_{\mathrm{IMG}}}$, often parameterized via $\log s_{u,i}$ for numerical stability.

$n_{u,j}$ The *variational mean* of $Q_{u,j}$; grouped into a mean matrix $N \in \mathbb{R}^{d \times d_{\mathrm{TXT}}}$.

$t_{u,j}$ The *variational standard deviation* of $Q_{u,j}$; grouped into a matrix $T \in \mathbb{R}^{d \times d_{\mathrm{TXT}}}$, likewise often parameterized via $\log t_{u,j}$.

d The dimensionality of the shared embedding space (i.e. the number of rows of both P and Q).

d_{IMG} The output dimension of the image encoder $\phi(\cdot)$.

d_{TXT} The output dimension of the text encoder $\psi(\cdot)$.

$\mathcal{N}(\mu, \sigma^2)$ The Gaussian (normal) distribution with mean μ and variance σ^2.

This mean field assumption (factorizing for each weight entry) allows us to derive closed-form expressions for KL divergences against simple Gaussian priors and to perform scalable stochastic gradient optimization using the reparameterization trick.

3.3 Evidence Lower Bound (ELBO)

We approximate the true posterior $p(P, Q \mid D)$ by the variational factorization $q(P, Q) = q(P)\, q(Q)$. The ELBO is

$$\begin{aligned}\mathcal{L}(m, s, n, t) &= \mathbb{E}_{q(P,Q)}\big[\log p(D \mid P, Q)\big] \;-\; \mathrm{KL}\big(q(P, Q) \,\|\, p(P, Q)\big) \\ &= \mathbb{E}_{q(P,Q)}\Big[\sum_{(x,y)\in D} \log p(y \mid x, P, Q)\Big] \;-\; \mathrm{KL}\big(q(P, Q) \,\|\, p(P, Q)\big). \end{aligned} \tag{11}$$

Factorization of the KL term. Because we assume independence

$$q(P, Q) = q(P)\, q(Q), \quad p(P, Q) = p(P)\, p(Q),$$

the joint KL can be written by its definition as

$$\begin{aligned}\mathrm{KL}\big(q(P, Q) \,\|\, p(P, Q)\big) &= \int q(P, Q)\, \log \frac{q(P, Q)}{p(P, Q)}\, dP\, dQ \\ &= \int q(P)\, q(Q)\, \Big[\log q(P) - \log p(P) + \log q(Q) - \log p(Q)\Big]\, dP\, dQ \\ &= \int q(P)\, \log \frac{q(P)}{p(P)}\, dP + \int q(Q)\, \log \frac{q(Q)}{p(Q)}\, dQ \\ &= \mathrm{KL}\big(q(P) \,\|\, p(P)\big) + \mathrm{KL}\big(q(Q) \,\|\, p(Q)\big).\end{aligned}$$

Thus the complexity penalty splits additively:

$$\mathrm{KL}\big(q(P, Q) \,\|\, p(P, Q)\big) = \mathrm{KL}\big(q(P) \,\|\, p(P)\big) + \mathrm{KL}\big(q(Q) \,\|\, p(Q)\big). \tag{12}$$

KL for P. With independent zero-mean Gaussian priors $p(P_{u,i}) = \mathcal{N}(0, \lambda^{-1})$, we have

$$\mathrm{KL}\big(q(P) \,\|\, p(P)\big) = \sum_{u=1}^{d} \sum_{i=1}^{d_{\mathrm{IMG}}} \mathrm{KL}\big(\mathcal{N}(m_{u,i}, s_{u,i}^2) \,\big\|\, \mathcal{N}(0, \lambda^{-1})\big),$$

and each scalar KL is

$$\mathrm{KL}\big(\mathcal{N}(m, s^2) \,\big\|\, \mathcal{N}(0, \lambda^{-1})\big) = \frac{1}{2}\big[\lambda s^2 + \lambda m^2 - 1 - \ln(\lambda s^2)\big]. \tag{13}$$

then we will sum over each for both P and Q

KL for Q. Analogously,

$$\mathrm{KL}\big(q(Q) \,\|\, p(Q)\big) = \sum_{u=1}^{d} \sum_{j=1}^{d_{\mathrm{TXT}}} \mathrm{KL}\big(\mathcal{N}(n_{u,j}, t_{u,j}^2) \,\big\|\, \mathcal{N}(0, \lambda^{-1})\big),$$

with each term given by (13) upon replacing $(m, s) \mapsto (n, t)$.

Putting everything into (11), the final ELBO reads

$$\begin{aligned}\mathcal{L}(m, s, n, t) = \mathbb{E}_{q(P,Q)}\Big[\sum_{(x,y)\in D} \log p(y \mid x, P, Q)\Big] \\ - \sum_{u=1}^{d} \sum_{i=1}^{d_{\mathrm{IMG}}} \mathrm{KL}\big(\mathcal{N}(m_{u,i}, s_{u,i}^2) \| \mathcal{N}(0, \lambda^{-1})\big) \\ - \sum_{u=1}^{d} \sum_{j=1}^{d_{\mathrm{TXT}}} \mathrm{KL}\big(\mathcal{N}(n_{u,j}, t_{u,j}^2) \| \mathcal{N}(0, \lambda^{-1})\big).\end{aligned} \tag{14}$$

This completes the explicit derivation of how the joint KL reduces to individual summations over P and Q due to the mean–field independence assumption.

3.4 Expected Log-Likelihood and Full ELBO Flow

InfoNCE Log-Likelihood. For a batch of n paired samples $\{(x_i, y_i)\}_{i=1}^n$, let

$$g_i \;=\; P\,\phi(x_i) \;\in\; \mathbb{R}^d, \qquad h_j \;=\; Q\,\psi(y_j) \;\in\; \mathbb{R}^d,$$

be the unnormalized embeddings, and

$$\hat{g}_i = \frac{g_i}{\|g_i\|}, \quad \hat{h}_j = \frac{h_j}{\|h_j\|},$$

their *unit-length normalized* versions. The InfoNCE conditional log-likelihood for predicting the correct text y_i from image x_i is

$$\begin{aligned} \log p_{\text{InfoNCE}}(y_i = i \mid x_i, P, Q) &= t\,\hat{g}_i^\top \hat{h}_i - \log \sum_{j=1}^n \exp\bigl(t\,\hat{g}_i^\top \hat{h}_j\bigr), \\ \log p_{\text{InfoNCE}}(x_i = i \mid y_i, P, Q) &= t\,\hat{h}_i^\top \hat{g}_i - \log \sum_{j=1}^n \exp\bigl(t\,\hat{h}_i^\top \hat{g}_j\bigr), \end{aligned} \tag{15}$$

where:

- $t > 0$ is the *learnable temperature* scaling the cosine logits.
- $\hat{g}_i^\top \hat{h}_j$ is the *cosine similarity* between image i and text j.
- The first line is the *image-to-text* loss; the second is the *text-to-image* symmetric term.

Summing over all n pairs yields the full expected log-likelihood under $q(P, Q)$:

$$\mathbb{E}_{q(P,Q)}\Bigl[\sum_{i=1}^n \log p_{\text{InfoNCE}}(y_i = i \mid x_i, P, Q)\Bigr] + \mathbb{E}_{q(P,Q)}\Bigl[\sum_{i=1}^n \log p_{\text{InfoNCE}}(x_i = i \mid y_i, P, Q)\Bigr].$$

Full ELBO with Variational Parameters. Combining the expected log-likelihood with the KL complexity penalties over both P and Q, the final ELBO to *maximize* is

$$\begin{aligned} \mathcal{L}(m, s, n, t) = \underbrace{\mathbb{E}_{q(P,Q)}\Bigl[\sum_{i=1}^n \bigl(\log p_{\text{InfoNCE}}(y_i{=}i \mid x_i) \;+\; \log p_{\text{InfoNCE}}(x_i{=}i \mid y_i)\bigr)\Bigr]}_{\text{Expected contrastive log-likelihood}} \\ \underbrace{- \sum_{u=1}^d \sum_{i=1}^{d_{\text{IMG}}} \text{KL}\bigl(\mathcal{N}(m_{u,i}, s_{u,i}^2) \,\|\, \mathcal{N}(0, \lambda^{-1})\bigr)}_{\text{KL penalty for } P} \\ \underbrace{- \sum_{u=1}^d \sum_{j=1}^{d_{\text{TXT}}} \text{KL}\bigl(\mathcal{N}(n_{u,j}, t_{u,j}^2) \,\|\, \mathcal{N}(0, \lambda^{-1})\bigr)}_{\text{KL penalty for } Q}. \end{aligned} \tag{16}$$

Here:

- $\{m_{u,i}, s_{u,i}\}$ are the *mean* and *std-dev* variational parameters for P.
- $\{n_{u,j}, t_{u,j}\}$ are the analogous parameters for Q.
- λ is the *prior precision* of the zero-mean Gaussian prior $p(\cdot) = \mathcal{N}(0, \lambda^{-1})$.

Maximizing $\mathcal{L}(m, s, n, t)$ via stochastic gradient ascent (using the reparameterization trick on $q(P)$ and $q(Q)$) yields the optimal variational approximation to the posterior over P and Q.

3.5 Optimization via Reparameterization

To optimize the variational parameters $(m,\ s)$ for P and $(n,\ t)$ for Q, we maximize the ELBO by sampling from the variational posterior using the reparameterization trick:

$$\varepsilon_{iu} \sim \mathcal{N}(0,1), \quad \zeta_{jl} \sim \mathcal{N}(0,1),$$
$$P_{iu} = m_{iu} + s_{iu}\,\varepsilon_{iu}, \qquad Q_{jl} = n_{jl} + t_{jl}\,\zeta_{jl}.$$

Here m_{iu} and n_{jl} are the means, and $s_{iu} = \exp(\frac{1}{2}\,\rho_{iu})$, $t_{jl} = \exp(\frac{1}{2}\,\tau_{jl})$ are the standard deviations (with ρ, τ their log-variances). With each sampled (P, Q), we compute the expected log-likelihood (InfoNCE or Sigmoid) and subtract the closed-form Gaussian KLs, then back-propagate through all operations to update (m, ρ, n, τ).

Algorithm 3 Mean–Field VI for BayesVLM

Input: paired dataset $\mathcal{D}$, encoders $\phi(\cdot), \psi(\cdot)$
Init: variational means $\{m_{iu}\}, \{n_{jl}\}$ and log-variances $\{\rho_{iu}\}, \{\tau_{jl}\}$
repeat
  Sample $\varepsilon_{iu}, \zeta_{jl} \sim \mathcal{N}(0,1)$
  Compute $s_{iu} = \exp(\frac{1}{2}\rho_{iu}),\ t_{jl} = \exp(\frac{1}{2}\tau_{jl})$
  Form samples $P_{iu} \leftarrow m_{iu} + s_{iu}\,\varepsilon_{iu}, \quad Q_{jl} \leftarrow n_{jl} + t_{jl}\,\zeta_{jl}$
  Compute embeddings $g_i = P\,\phi(x_i),\ h_j = Q\,\psi(x_j)$
  Evaluate contrastive log-likelihood $\sum_i \log p(y_i \,|\, x_i, P, Q)$ via Eq. (**??**) (or sigmoid loss)
  Compute KLs $\mathrm{KL}(q(P)\|p(P))$ and $\mathrm{KL}(q(Q)\|p(Q))$ via Eq. (**??**)
  $\mathcal{L} \leftarrow$ expected log-likelihood $-$ (KL_P+KL_Q)
  Update $\{m, \rho, n, \tau\} \leftarrow \{m, \rho, n, \tau\} + \eta\,\nabla\mathcal{L}$
until convergence

3.6 2nd Approach for VI: Variational Inference using Matrix Normal Distribution(considering correlation between each entries of P and Q)

Vision-Language Models (VLMs) like CLIP embed images and text into a shared d-dimensional vector space via linear projection layers $P \in \mathbb{R}^{d \times p_{\mathrm{IMG}}}$ and $Q \in \mathbb{R}^{d \times p_{\mathrm{TXT}}}$. After pre-training, we seek post-hoc uncertainty estimates over only these heads. We assume P and Q are independent blocks but model correlations within each via matrix-variate Gaussian posteriors unlike above mean fields VI approach.

3.6.1 Basic Notation

- $\phi(x) \in \mathbb{R}^{p_{\mathrm{IMG}}}$: image encoder output
- $\psi(y) \in \mathbb{R}^{p_{\mathrm{TXT}}}$: text encoder output
- $P \in \mathbb{R}^{d \times p_{\mathrm{IMG}}}$: image projection head
- $Q \in \mathbb{R}^{d \times p_{\mathrm{TXT}}}$: text projection head
- $\Theta = [P \mid Q] \in \mathbb{R}^{d \times (p_{\mathrm{IMG}} + p_{\mathrm{TXT}})}$: concatenated heads
- $D = \{(x_i, y_i)\}_{i=1}^{N}$: dataset of image-text pairs

3.6.2 Matrix-Normal Variational Family

We assume separate matrix-normal variational distributions for P and Q:

$$q(P) = \mathcal{MN}(M_P, U_P, V_P), \tag{17}$$
$$q(Q) = \mathcal{MN}(M_Q, U_Q, V_Q), \tag{18}$$

where for a generic $H \in \{P, Q\}$,

$$q(H) = \mathcal{MN}(M_H, U_H, V_H) \quad \Longleftrightarrow \quad \mathrm{vec}(H) \sim \mathcal{N}(\mathrm{vec}(M_H),\ V_H \otimes U_H). \tag{19}$$

where:

- $M_H \in \mathbb{R}^{d \times p}$ is the mean (initialized to H_{MAP}).
- $U_H \in \mathbb{R}^{d \times d}$ is the row-covariance (output dimensions).
- $V_H \in \mathbb{R}^{p \times p}$ is the column-covariance (input dimensions).

3.6.3 Evidence Lower Bound

We assume matrix-normal priors over each projection head:

$$p(P) = \mathcal{MN}\big(0,\ \tau^{-1}I,\ \tau^{-1}I\big),$$
$$p(Q) = \mathcal{MN}\big(0,\ \tau^{-1}I,\ \tau^{-1}I\big).$$

The ELBO for block-independent posteriors is

$$\mathcal{L} = \underbrace{\mathbb{E}_{q(P)q(Q)}\big[\log p(D \mid P, Q)\big]}_{(A)} - \underbrace{\mathrm{KL}\big(q(P) \,\|\, p(P)\big)}_{(B_P)} - \underbrace{\mathrm{KL}\big(q(Q) \,\|\, p(Q)\big)}_{(B_Q)}. \tag{20}$$

3.6.4 Term (B): KL Divergence

For each head $H \in \{P, Q\}$ with size $d \times p$, we have

$$q(H) = \mathcal{MN}(M_H, U_H, V_H), \quad p(H) = \mathcal{MN}\big(0,\ \tau^{-1}I,\ \tau^{-1}I\big).$$

By vectorizing, $\mathrm{vec}(H) \sim \mathcal{N}(\mathrm{vec}(M_H), V_H \otimes U_H)$ and the prior $\Sigma_0 = \tau^{-1}I \otimes \tau^{-1}I$. Applying the multivariate Gaussian KL formula yields

$$\mathrm{KL}\big(q(H) \,\|\, p(H)\big) = \tfrac{1}{2}\Big[\mathrm{vec}(M_H)^\top(\tau\, I \otimes \tau\, I)\,\mathrm{vec}(M_H) + \mathrm{tr}(U_H)\,\mathrm{tr}(V_H) - dp - p\ln|U_H| - d\ln|V_H| + (d+p)\ln\tau\Big]. \tag{21}$$

3.6.5 Detailed Derivation for KL Divergence term for Matrix Normal distribution

We derive the KL divergence between two matrix-normal distributions in full detail, then specialize to our variational posterior $q(H)$ and prior $p(H)$.

[KL for Matrix-Normal Distributions] Let

$$P: \quad X \sim \mathcal{MN}(M_1, U_1, V_1), \quad Q: \quad X \sim \mathcal{MN}(M_2, U_2, V_2),$$

be two matrix-normal distributions over $X \in \mathbb{R}^{n \times p}$. Then

$$\mathrm{KL}(P\|Q) = \frac{1}{2}\Big[\mathrm{vec}(\Delta M)^T(V_2^{-1} \otimes U_2^{-1})\,\mathrm{vec}(\Delta M) + \mathrm{tr}\big((V_2^{-1}V_1) \otimes (U_2^{-1}U_1)\big) - np + p\ln\frac{|U_2|}{|U_1|} + n\ln\frac{|V_2|}{|V_1|}\Big], \tag{22}$$

where $\Delta M = M_2 - M_1$.

First, recall the vec–normal equivalence:

$$X \sim \mathcal{MN}(M, U, V) \quad \Longleftrightarrow \quad \mathrm{vec}(X) \sim \mathcal{N}\big(\mathrm{vec}(M),\ \Sigma\big), \quad \Sigma = V \otimes U.$$

The KL divergence between D-dimensional Gaussians $\mathcal{N}(\mu_1, \Sigma_1)$ and $\mathcal{N}(\mu_2, \Sigma_2)$ is

$$\mathrm{KL}(\mathcal{N}(\mu_1, \Sigma_1)\|\mathcal{N}(\mu_2, \Sigma_2)) = \frac{1}{2}\Big[(\mu_2 - \mu_1)^T\Sigma_2^{-1}(\mu_2 - \mu_1) + \mathrm{tr}(\Sigma_2^{-1}\Sigma_1) - D + \ln\frac{|\Sigma_2|}{|\Sigma_1|}\Big].$$

Setting $\mu_1 = \mathrm{vec}(M_1)$, $\mu_2 = \mathrm{vec}(M_2)$, $\Sigma_1 = V_1 \otimes U_1$, $\Sigma_2 = V_2 \otimes U_2$, and $D = np$, we obtain:

1. Quadratic term

$$(\mu_2 - \mu_1)^T\Sigma_2^{-1}(\mu_2 - \mu_1) = \mathrm{vec}(\Delta M)^T\,(V_2^{-1} \otimes U_2^{-1})\,\mathrm{vec}(\Delta M).$$

2. Trace term Using $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$ and $\mathrm{tr}(A \otimes B) = \mathrm{tr}(A)\,\mathrm{tr}(B)$,

$$\mathrm{tr}(\Sigma_2^{-1}\Sigma_1) = \mathrm{tr}(V_2^{-1}V_1)\,\mathrm{tr}(U_2^{-1}U_1) = \mathrm{tr}\big((V_2^{-1}V_1) \otimes (U_2^{-1}U_1)\big).$$

3. Log-determinant term Since $|A \otimes B| = |A|^m\ |B|^n$ for $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{m \times m}$,

$$\ln\frac{|\Sigma_2|}{|\Sigma_1|} = p\ln\frac{|U_2|}{|U_1|} + n\ln\frac{|V_2|}{|V_1|}.$$

Substituting into the multivariate KL formula yields exactly (22).

Now specialize to our VI setup. For each head $H \in \{P, Q\}$ of size $d \times p$:

$$q(H) = \mathcal{MN}(M_H, U_H, V_H), \quad p(H) = \mathcal{MN}(0, \tau^{-1}I, \tau^{-1}I).$$

Here $M_1 = 0$, $U_1 = V_1 = \tau^{-1}I$, and $M_2 = M_H,\ U_2 = U_H,\ V_2 = V_H$, with $n = d,\ p =$ input dimension. Plugging these into (22) gives

$$\mathrm{KL}\big(q(H)\|p(H)\big) = \frac{1}{2}\Big[\tau\,\mathrm{vec}(M_H)^T\mathrm{vec}(M_H) + \mathrm{tr}(U_H)\,\mathrm{tr}(V_H) - dp - p\ln|U_H| - d\ln|V_H| + (d+p)\ln\tau\Big]. \tag{23}$$

This closed-form expression is then used directly in the ELBO (20).

3.6.6 Expanded ELBO with Detailed KL Terms

We place block-independent matrix-normal priors over each projection head:

$$p(P) = \mathcal{MN}\big(0,\ \tau^{-1}I,\ \tau^{-1}I\big), \quad p(Q) = \mathcal{MN}\big(0,\ \tau^{-1}I,\ \tau^{-1}I\big).$$

The ELBO then expands to

$$\mathcal{L} = \underbrace{\mathbb{E}_{q(P)\,q(Q)}\big[\log p(D \mid P, Q)\big]}_{(A)} - \underbrace{\mathrm{KL}\big(q(P)\,\|\,p(P)\big)}_{(B_P)} - \underbrace{\mathrm{KL}\big(q(Q)\,\|\,p(Q)\big)}_{(B_Q)}.$$

$$(A) = \mathbb{E}_{q(P)\,q(Q)}\Big[-L_{\mathrm{InfoNCE}}(P, Q; D) \quad \text{or} \quad -L_{\mathrm{SigLIP}}(P, Q; D)\Big],$$

$$L_{\mathrm{InfoNCE}} = -\frac{1}{2N}\sum_{i=1}^{N}\log\frac{\exp\big(t\,g_i^\top h_i\big)}{\sum_{j=1}^{N}\exp\big(t\,g_i^\top h_j\big)},$$

$$L_{\mathrm{SigLIP}} = -\frac{1}{N}\sum_{i,j=1}^{N}\log\big(\sigma\big(z_{ij}(-t\,g_i^\top h_j + b)\big)\big).$$

$$(B_P) \quad \mathrm{KL}\big(q(P)\|p(P)\big) = \frac{1}{2}\Big[\underbrace{\mathrm{vec}(M_P)^\top(\tau I \otimes \tau I)\,\mathrm{vec}(M_P)}_{\text{(i) quad.}} + \underbrace{\mathrm{tr}(U_P)\,\mathrm{tr}(V_P)}_{\text{(ii) trace}} - d\,p_{\mathrm{IMG}} \underbrace{- p_{\mathrm{IMG}}\ln|U_P|}_{\text{(iii) row log-det}} - \underbrace{d\ln|V_P|}_{\text{(iv) col log-det}} + (\,d + p_{\mathrm{IMG}}\,)\ln\tau\Big],$$

$$(B_Q) \quad \mathrm{KL}\big(q(Q)\|p(Q)\big) = \frac{1}{2}\Big[\underbrace{\mathrm{vec}(M_Q)^\top(\tau I \otimes \tau I)\,\mathrm{vec}(M_Q)}_{\text{(i) quad.}} + \underbrace{\mathrm{tr}(U_Q)\,\mathrm{tr}(V_Q)}_{\text{(ii) trace}} - d\,p_{\mathrm{TXT}} \underbrace{- p_{\mathrm{TXT}}\ln|U_Q|}_{\text{(iii) row log-det}} - \underbrace{d\ln|V_Q|}_{\text{(iv) col log-det}} + (\,d + p_{\mathrm{TXT}}\,)\ln\tau\Big]. \tag{24}$$

where

(A) Expected negative contrastive log-likelihood under $q(P)\,q(Q)$. We can Choose InfoNCE or SigLIP.

(i) Quadratic term: penalizes deviation of M_H from zero under precision τ.

(ii) Trace term: aligns posterior covariances U_H, V_H with the prior scale.

(iii) and (iv) Row/column log-determinant terms: encourage covariance volumes close to $\tau^{-1}I$.

- The constant $(d + p_H)\ln\tau$ arises from the prior normalization.

3.7 Optimization via Adam

To optimize the variational parameters $\{M_H, U_H, V_H\}$ for each projection head $H \in \{P, Q\}$, we employ the Adam optimizer, an adaptive learning rate optimization algorithm well-suited for stochastic objectives such as the Evidence Lower Bound (ELBO).

3.7.1 Adam Optimization Algorithm

The Adam optimizer maintains exponentially decaying averages of past gradients (m_t) and squared gradients (v_t), which are estimates of the first and second moments, respectively. The update rules for each parameter θ at iteration t are as follows:

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla_\theta \mathcal{L}_t \\
v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_\theta \mathcal{L}_t)^2 \\
\hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\
\hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\
\theta_t &= \theta_{t-1} + \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}
\end{aligned}
$$

Here:

- α is the learning rate.
- β_1 and β_2 are the exponential decay rates for the moment estimates.
- ϵ is a small constant to prevent division by zero.
- $\nabla_\theta \mathcal{L}_t$ is the gradient of the ELBO with respect to parameter θ at iteration t.

3.7.2 Application to Variational Parameters

For each variational parameter:

- **Mean Matrix (M_H)**: The gradient is computed as:

  $$\nabla_{M_H} \mathcal{L} = \mathbb{E}_q[\nabla_H \log p(D \mid H)] - \tau M_H$$

  where τ is a regularization coefficient, and the expectation is approximated using Monte Carlo samples via the reparameterization trick.
- **Row Covariance (U_H)** and **Column Covariance (V_H)**: The gradients involve derivatives of the log-determinant and trace terms from the KL divergence between the variational posterior and the prior. These are computed as:

  $$\nabla_{U_H} \mathcal{L} = \frac{1}{2} U_H^{-1} - \frac{1}{2} U_0^{-1} + \text{additional terms from the expected log-likelihood}$$

  $$\nabla_{V_H} \mathcal{L} = \frac{1}{2} V_H^{-1} - \frac{1}{2} V_0^{-1} + \text{additional terms from the expected log-likelihood}$$

  where U_0 and V_0 are the prior covariances.

3.7.3 Implementation Details

In practice, the optimization proceeds as follows:

1. **Initialization**: Set initial values for M_H, U_H, and V_H, and initialize m_0 and v_0 to zero for each parameter.
2. **Iterative Updates**:
   - Sample a mini-batch of data D_b.

- Use the reparameterization trick to sample from $q(H)$:

$$H = M_H + L_{U_H} \cdot \Xi \cdot L_{V_H}^\top$$

where Ξ is a standard normal matrix, and L_{U_H} and L_{V_H} are the Cholesky decompositions of U_H and V_H, respectively.
- Compute the stochastic gradients $\nabla_\theta \mathcal{L}_t$ for each parameter θ.
- Update the first and second moment estimates m_t and v_t.
- Compute the bias-corrected estimates $\hat{m}_t$ and $\hat{v}_t$.
- Update the parameters using the Adam update rule.

3.7.4 Hyperparameter Settings

set values for the Adam hyperparameters(for e.g):

- Learning rate $\alpha = 0.001$
- $\beta_1 = 0.9$
- $\beta_2 = 0.999$
- $\epsilon = 10^{-8}$

These values may be tuned based on validation performance.

3.7.5 Advantages of Using Adam

Adam combines the benefits of two other extensions of stochastic gradient descent: adaptive learning rates (as in AdaGrad) and momentum (as in RMSProp). This makes it well-suited for problems with sparse gradients and noisy objectives, such as variational inference in deep models.

3.8 Uncertainty Propagation and Evaluation

Given input features $\phi(x)$ and $\psi(y)$,

$$g = P\,\phi(x) \sim \mathcal{N}(\mu_g, \Sigma_g), \qquad \mu_g = M_P \phi(x),\ \Sigma_g = (\phi(x)^\top V_P \phi(x))\, U_P, \tag{25}$$

$$h = Q\,\psi(y) \sim \mathcal{N}(\mu_h, \Sigma_h). \tag{26}$$

The cosine similarity $S = \frac{g^\top h}{\|g\|\|h\|}$ is approximated as

$$S \sim \mathcal{N}(\mathbb{E}[S], \mathrm{Var}[S]) \tag{27}$$

using moment formulas (see main text). We then use $\mathbb{E}[S]$ for classification scores and $\sqrt{\mathrm{Var}[S]}$ for calibration.

4 Description of Tools / Software Used

- **Python, PyTorch**: For model loading, implementation, and experimentation.
- **OpenCLIP, SigLIP Weights**: Pre-trained models used in the experiments.
- **Home-art Dataset**: Used for estimating the Hessian matrix.
- **Home-art Dataset**: Used for finetuning and activelearning and posterior.
- **NumPy, SciPy**: For matrix operations and posterior computation.
- **Matplotlib, Seaborn**: For plotting calibration and performance curves.
- **Jupyter Notebooks**: For prototyping and experimentation.
- **GitHub, Google Colab, HPC**: For collaboration, version control, and high-performance computations.

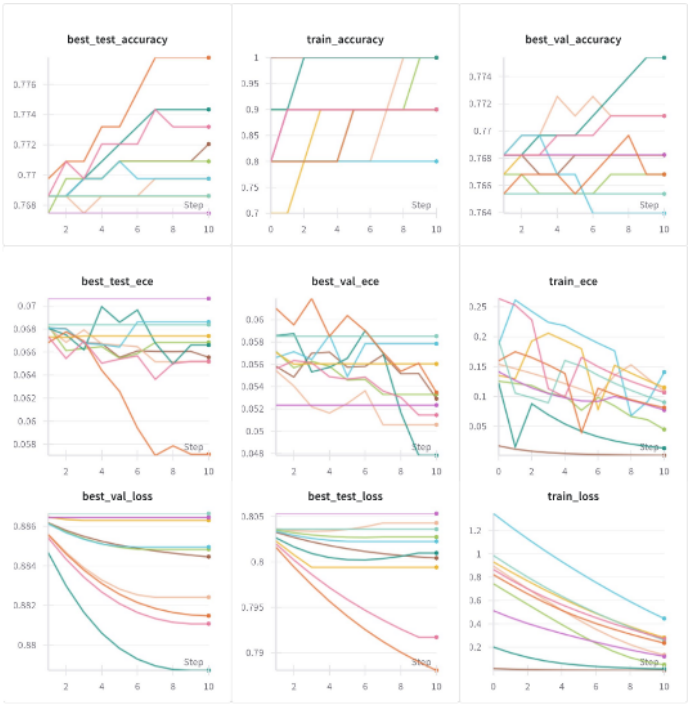

Figure 1: Loss and accuracy using KNN support set creation

SNN

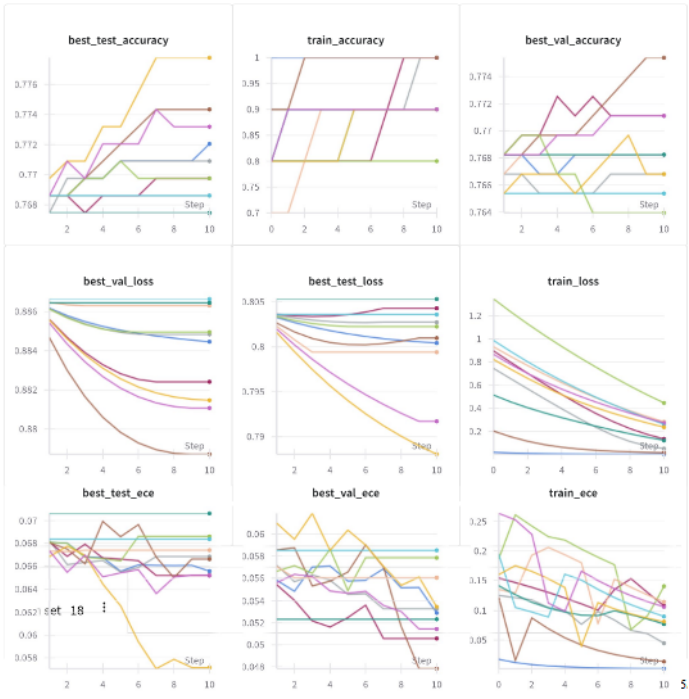

Figure 2: Loss and accuracy using SNN support set creation

5 Experimental Results

The activelearning was run initially using the paper-proposed KNN support set creation. The results are as shown

Then it was swapped to an SNN.

While there is not significant change in this dataset, it can prove to be useful in cases where density is high near the query point and outliers are plentiful. SNN can outperform KNN in scenarios where the data contains significant noise, outliers, or clusters of varying density. Unlike KNN, which may select redundant or noisy neighbors due to its reliance on raw distance, SNN evaluates the overlap in neighbor lists, making it more robust to high-dimensional noise and less likely to include outliers. This connectivity-based approach ensures that support samples are drawn from genuinely dense and meaningful regions of the embedding space, promoting diversity and reducing redundancy. As a result, SNN is particularly advantageous in complex or heterogeneous datasets where KNN's strict proximity criterion may fail to capture the true structure of the data.

5.1 Evaluation Metrics

We employed several metrics to assess both predictive performance and uncertainty quality:

- **Top-1 Accuracy**: For zero-shot classification.
- **Expected Calibration Error (ECE)**: Measures how well predicted confidence matches observed accuracy.
- **Active Learning Performance**: Accuracy as a function of labeled sample count, using acquisition strategies like entropy and BALD.

5.2 Results Summary

6 Things That We Learned While Doing the Project

- **Variational Inference for Posterior:** By modeling the posterior with variational inference instead of the Laplace approximation, we gained flexibility in approximating complex(1st approach using Factorised Gaussian and 2nd Using Matrix Normal),we learned about Kronecker Algebra and Matrix Normal Distribution. This approach allowed us to better capture uncertainty in the vision-language embedding space, especially under domain shift. However, we observed that variational inference can be more computationally demanding and requires careful tuning of the variational family and optimization parameters.
- **Replacing KNN with SNN in Active Learning:** We replaced the traditional k-nearest neighbor (KNN) selection with a Shared Nearest Neighbor strategy. k-Nearest Neighbors (KNN) directly targets each test example by selecting its k most similar support candidates in the vision-language embedding space, ensuring the support set is immediately relevant to the downstream queries and improving sample efficiency in adaptation. Unlike Shared Nearest Neighbor (SNN), which builds a global neighbor-overlap graph and may overlook isolated test regions, KNN guarantees per-sample coverage, so no test point goes unrepresented. KNN also has a single, intuitive hyperparameter (k) and can leverage highly optimized approximate nearest-neighbor libraries for speed. This simplicity reduces both implementation complexity and the risk of misconfigured thresholds, leading to more predictable and reproducible active-learning performance.
- **Trade-offs and Practical Insights:** Overall, we learned that while advanced Bayesian and neural selection methods can offer measurable gains in uncertainty quantification and active learning, they also introduce new computational and optimization challenges. Balancing model flexibility, efficiency, and stability remains a key consideration in practical deployments.

7 Possible Future Work

- **Cross-Modal Uncertainty Propagation:** Final projection layers are the focus of present work, but applying uncertainty quantification to attention mechanisms within transformer

architectures might be able to reflect compositional uncertainties in vision-language interactions. Efficient Hessian approximations for self-attention layers would be needed for this.

- **Streaming Active Learning:** Constructing an online Laplace approximation which is incrementally updating Kronecker factors might facilitate real-time active learning. This would facilitate continuous model adaptation with applications in robotic systems and augmented reality.
- **Closed-Source Model Adaptation:** Creating proxy datasets or synthetic data generation techniques to estimate Hessians when original training data is unavailable. This could extend BayesVLM's applicability to commercial VLMs like GPT-4V where training data is inaccessible.
- **Multimodal Retrieval Applications:** Generalizing ProbCosine similarity to compositional retrieval tasks by representing joint uncertainties over image regions and text tokens. This may enhance reliability in medical imaging and autonomous driving tasks.
- **Hierarchical Uncertainty Quantification:** Constructing nested Laplace approximations that disentangle object-level and pixel-level uncertainties. This may facilitate more fine-grained uncertainty visualization and editing in generative VLMs.

8 Team Member Contributions

- **J Kartik Sai**: SNN implementation and Active learning.
- **Deepak Chaurasia**: Meanfield VI mathematics and formulation
- **Riya Mittal**: Active learning loop, dataset processing, writing results.

Disclaimer

We declare that this project was undertaken solely for the purposes of CS772 and has not been submitted, credited, or used for evaluation in any other course, internship, or academic activity. All work presented here is original within the scope of this course.

The Variational inference part of the implementation was not done due to constraints with time and computational resources. Although we have given a proposal of applying VI in this project, we were unfortunately unable to implement due to aforementioned reasons.